

SOLUTION REQUIREMENTS

Functional & Non-functional Requirements
for the HDI Prediction System

Project Design Phase-II: Solution Requirements (Functional & Non-functional)

HDI Prediction System — Machine Learning Powered Human Development Index Prediction Platform

Project	A Comprehensive Measure of Well-Being (HDI Prediction System)
Authors	Paruchuri Venkatesh; Rokkam Guna Sekhar
Phase	Design Phase-II — Solution Requirements
Target Stack	Python, Flask, Scikit-Learn, SQLite, SQLAlchemy, Bootstrap, Flask-Login, Pickle

This document defines the **functional** and **non-functional** requirements for the HDI Prediction System as part of Phase-II (Solution Requirements). It is implementation-focused and aligned to the existing project design and capabilities: secure user authentication, HDI prediction from socio-economic indicators, dashboard analytics, prediction history management, and secure data handling.

1. Project Background

The Human Development Index (HDI) is a widely accepted measure of national development because it incorporates multiple dimensions of well-being—health, education, and standard of living—rather than relying solely on economic output. Conventional HDI assessment depends on periodically published reports (e.g., UNDP), which introduces delays and limits rapid forecasting for policy and research use cases.

The HDI Prediction System addresses this gap through a Machine Learning-powered web application that estimates a country's HDI score from key socio-economic indicators in near real time. The system operationalizes an ML model (Linear Regression, serialized with Pickle) behind a Flask-based web interface and stores user and prediction data in SQLite via SQLAlchemy ORM.

Key intent — Provide a secure, accessible, and reproducible platform that supports data-driven HDI estimation, result categorization (Low/Medium/High/Very High), and basic analytics over prediction history.

2. Objective of Phase-II (Solution Requirements)

Phase-II defines the **complete requirements baseline** for the HDI Prediction System to enable consistent implementation, testing, and evaluation. Specifically, it establishes: (a) what the system must do (functional requirements), (b) the quality attributes it must meet (non-functional requirements), (c) constraints and assumptions that shape implementation, and (d) measurable acceptance criteria for verification and sign-off.

3. Scope

In scope:

- Secure user registration, login, logout, and session management using Flask-Login.
- HDI prediction from user-provided indicators (Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, GNI per Capita).
- Result presentation including predicted HDI score and UNDP-aligned category (Low/Medium/High/Very High).
- Prediction persistence to a relational store (SQLite) and retrieval via SQLAlchemy ORM.
- Dashboard analytics (counts, averages, distribution visualization, recent activity).
- Prediction history listing, inspection, and deletion (user-scoped).
- Input validation and secure handling of user data and credentials.

Out of scope (Phase-II baseline; may be future enhancements):

- Live UN/World Bank data ingestion and automated dataset refresh pipelines.
- Advanced model families (e.g., LSTM, XGBoost), explainability (e.g., SHAP), and automated recommendation engines.
- Public REST API for external consumers, mobile application, multi-language UI, and cloud-native scaling beyond single-node deployment.

4. System Overview

System type — ML-enabled web application for HDI estimation and analytics.

Primary users — Students, researchers, policy analysts, NGOs, and government stakeholders requiring rapid HDI estimation and comparative analysis support.

High-level components:

- **Presentation Layer** — Bootstrap-based UI templates rendered via Jinja2.
- **Application Layer** — Flask application (routes/controllers), authentication integration, request handling, and templating.
- **Business Logic** — validation, categorization rules, analytics aggregation, and history management.
- **Machine Learning Layer** — preprocessing and inference using a trained Scikit-Learn Linear Regression model loaded from Pickle.
- **Data Access Layer** — SQLAlchemy ORM models for users and predictions persisted in SQLite.

Input	Processing	Output
Country name (reference label) + 4 numeric indicators	Validate → scale/transform (as per trained pipeline) → ML inference → categorize	HDI score (0–1) + category + stored prediction record + dashboard updates

6. Non-Functional Requirements

5. Functional Requirements

Functional requirements define the observable system behavior. Each requirement is testable and aligned to the project’s current implementation approach (Flask, Flask-Login, SQLAlchemy, Scikit-Learn, and Pickle).

ID	Requirement	Rationale / Notes
FR-01	The system shall allow a user to register an account using a unique username (or email) and password.	Uniqueness enforced at the database level.
FR-02	The system shall hash and salt user passwords prior to persistence and shall never store plain-text passwords.	Use Werkzeug-compatible hashing.
FR-03	The system shall authenticate registered users via a login form and create an active session using Flask-Login.	Supports protected routes and session management.
FR-04	The system shall provide logout functionality that invalidates the active user session.	Prevents unauthorized session reuse.
FR-05	The system shall restrict access to prediction, dashboard, and history views to authenticated users only.	Enforced through route protection (login required).
FR-06	The system shall accept prediction inputs: Country Name, Life Expectancy (years), Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita (USD).	Matches the dataset feature set used for model training.
FR-07	The system shall validate inputs on both client-side and server-side, rejecting missing fields and non-numeric values for numeric indicators.	Prevents invalid inference requests.
FR-08	The system shall enforce configurable numeric range checks and display actionable validation errors.	Example bounds: indicators must be > 0.
FR-09	Upon valid submission, the system shall load the trained ML model from a serialized artifact (Pickle) and generate an HDI score prediction.	Artifact must be deployed with controlled access.
FR-10	The system shall return the predicted HDI score within [0.000, 1.000], rounded for display while preserving full precision for storage.	UI formatting does not alter stored values.
FR-11	The system shall compute and display a development category based on the predicted HDI score: Low, Medium, High, or Very High.	Thresholds align with UNDP-style category conventions as implemented.
FR-12	The system shall persist each successful prediction to SQLite via SQLAlchemy, storing: user association,	Enables history and analytics.

	timestamp, country name, indicators, predicted score,	
FR-13	The system shall provide a Prediction History view listing user-scoped predictions in reverse chronological order.	Cross-user access must be prevented by query scoping.
FR-14	The system shall allow an authenticated user to delete their own prediction records and reflect deletion in history and dashboard aggregates.	Deletion must not affect other users.
FR-15	The system shall provide a Dashboard view showing user-scoped analytics including total predictions, average HDI score, distribution visualization, and recent activity.	Charts may be generated using Matplotlib and rendered in the UI.
FR-16	The system shall prevent sensitive data exposure by not logging credential values and by protecting authenticated routes.	Support secure handling and basic auditability.
FR-17	The system shall handle model/inference failures (e.g., missing or corrupted Pickle artifact) with a controlled error message and without partial database writes.	Ensures consistency and resilience.

Implementation note — Model inference must use the same preprocessing assumptions as training (e.g., scaling). If preprocessing is not embedded in the serialized artifact, the application must apply an equivalent preprocessing step before calling *predict*.

Non-functional requirements define the quality attributes for the system. Targets are set to be realistic for a Flask + SQLite deployment and consistent with an academic implementation while remaining measurable.

Category	Requirement	Target / Measure
Performance	The system shall return an HDI prediction response for a valid request within an acceptable response time under normal load.	< 1.0 s server-side for inference and persistence on a typical development machine; UI render within 2.0 s total.
Availability	The system shall remain operational for interactive single-node usage with graceful handling of recoverable errors.	Clear error pages; no application crashes for invalid user inputs.
Security	The system shall protect authenticated routes and ensure user-scoped access control on history and dashboard data.	Unauthorized users receive redirect to login; cross-user data access is prevented by query scoping.
Security	The system shall enforce secure credential practices (hashed passwords) and session protections.	Password hashing enabled; session cookie configured for HTTPOnly; CSRF protection recommended for forms.
Data Integrity	The system shall ensure atomic persistence of prediction records.	Either the full prediction record is committed or none is (transactional behavior via SQLAlchemy).
Usability	The system shall provide clear form guidance, validation feedback, and readable presentation of results and categories.	Field-level validation messages; result card includes score and category.
Maintainability	The codebase shall be modular and testable (separation of concerns between routes, services, models, and ML artifacts).	Use Flask app factory / blueprints where applicable; unit tests via pytest for core logic.
Portability	The application shall run in a standard Python virtual environment with pinned dependencies.	Reproducible environment via requirements file; local execution without external services.
Privacy	The system shall limit stored data to the minimum necessary for functionality and avoid storing	User profile minimal; prediction data limited to indicators and timestamps.

7. Constraints

- **Technology constraint** — The implementation shall use Python and Flask for the web layer, Scikit-Learn for the ML model, SQLite + SQLAlchemy for persistence, Bootstrap for UI styling, Flask-Login for authentication, and Pickle for model serialization, consistent with the current project design.
- **Deployment constraint** — The baseline deployment is single-node (local or single server) without mandatory cloud services (AWS/Azure) and without containerization (Docker) as a hard requirement for Phase-II.
- **Model constraint** — The baseline predictive algorithm is Linear Regression with an expected performance consistent with prior evaluation ($R^2 \approx 0.939$, $RMSE \approx 0.025$ on the project dataset). Phase-II does not require replacing the model family.
- **Data constraint** — Predictions rely on user-provided indicators; live data ingestion is not required for Phase-II requirements satisfaction.

8. Assumptions

- Users provide development indicators in compatible units (years for education and life expectancy; USD for GNI per Capita) and understand that outputs are estimates.
- The serialized model artifact and any required preprocessing parameters are versioned and deployed together with the application.
- SQLite is sufficient for the expected scale (academic/demo use, limited concurrent users).
- The system operates over HTTP(S); for production-like evaluation, HTTPS termination is assumed to be handled by the hosting environment or reverse proxy.

10. Conclusion

This Phase-II requirements document establishes an implementation-aligned baseline for the HDI Prediction System, specifying functional behavior (authentication, ML-based HDI prediction, history management, and dashboard analytics) and non-functional quality targets (security, performance, usability, and maintainability). The constraints and assumptions reflect the current project architecture using Flask, Scikit-Learn, SQLite/SQLAlchemy, Bootstrap, Flask-Login, and Pickle serialization.

Upon satisfying the acceptance criteria, the project will be positioned for Phase-III activities such as extended evaluation, optional feature expansion (e.g., explainable AI, live data integration), and deployment hardening (e.g., containerization and cloud hosting) without altering the core validated requirements baseline.